IDE Square LLM

Studio Projektowe

Autorzy

Małgorzata Duda malgorzatad@student.agh.edu.pl
Jakub Macioł jmaciol@student.agh.edu.pl

EAIiIB
Katedra Informatyki Stosowanej
Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie

Kraków, Polska
25 stycznia 2026

Spis treści

1 Wybór technologii **3**

2 Architektura systemu **4**
2.1 Przegląd komponentów 4
2.2 Multi-Agent System (BOT) 4
2.3 IDE Square Client 4
2.4 Komunikacja BOT ↔ IDE 4

3 Szczegółowy opis workflow **5**
3.1 Orkiestracja agentów 5
3.2 Sekwencja przetwarzania 5
3.3 Przekazywanie danych między agentami 5
3.3.1 LLM Agent → Logic Agent 5
3.3.2 Logic Agent → Class Agent / State Agent 5
3.3.3 Class Agent ↔ Prover Agent 6
3.3.4 Przejście do Verifier Agent 6
3.4 Pętla iteracyjna 6
3.5 Algorytm orkiestracji 7
3.5.1 Główna pętla workflow 7
3.5.2 Algorytm decyzyjny routingu 7
3.6 Wynik końcowy 7

4 Agenty **9**
4.1 Komponenty systemu 9
4.2 Diagram przepływu 10
4.3 LLM Agent 10
4.4 Logic Agent 13
4.5 State Agent 15
4.6 Class Agent 16
4.7 Prover Agent 17
4.8 Verifier Agent 18
4.9 Logi 19

5 Integracja z IDE **20**
5.1 Architektura serwisu 20
5.2 Integracja po stronie IDE 20
5.3 Model komunikacji 20
5.3.1 Protokół wiadomości 20
5.3.2 Struktura odpowiedzi końcowej 21
5.3.3 Sesje i trwałość 21
5.3.4 Przepływ komunikacji 21

6 Eksperymenty **22**
6.1 Eksperyment 1 22
6.2 Eksperyment 2 23
6.3 Eksperyment 3 25
6.4 Eksperyment 4: Porównanie podejść 26
6.4.1 Podejścia 26

6.4.2 Metryki oceny . . . 27
6.4.3 Dane testowe . . . 27
6.4.4 Przypadek 3: user_account . . . 29
6.4.5 Przypadek 4: ticket_support . . . 29
6.4.6 Przypadek 5: payment_processing . . . 30
6.4.7 Przypadek 6: inventory_item . . . 30
6.4.8 Podsumowanie wyników . . . 31
6.5 Eksperyment 5: Ablacja algorytmu . . . 31
6.5.1 Warianty ablacyjne . . . 31
6.5.2 Metryki . . . 31
6.5.3 Wyniki eksperymentu . . . 32
6.5.4 Wpływ komponentów . . . 32
6.5.5 Wyniki szczegółowe per model . . . 32
6.5.6 Analiza wyników . . . 32
6.5.7 Wnioski . . . 33
6.6 Eksperyment 6: Dokładność dopełniania kwadratu opozycji . . . 33
6.6.1 Kwadrat logiczny . . . 33
6.6.2 Scenariusze testowe . . . 33
6.6.3 Przypadki testowe . . . 33
6.6.4 Wyniki eksperymentu . . . 34
6.6.5 Szczegółowe statystyki . . . 34
6.6.6 Trend dokładności . . . 34
6.6.7 Analiza wyników . . . 34
6.6.8 Wnioski . . . 34

7 Wnioski **36**
7.1 Skuteczność architektury wieloagentowej . . . 36
7.2 Odporność na jakość danych wejściowych . . . 36
7.3 Potencjał rozwoju . . . 36

1 Wybór technologii

Do implementacji systemu agentowego wybraliśmy LangGraph jako główny framework. Decyzja ta wynika z kilku istotnych zalet tego narzędzia. Przede wszystkim LangGraph zapewnia natywne wsparcie dla złożonych przepływów, w tym pętli iteracyjnych i warunków decyzyjnych, co pozwala na efektywną współpracę agentów, na przykład VerifierAgent może wielokrotnie konsultować się z LogicAgent, StateAgent i ProverAgent aż do osiągnięcia oczekiwanej spójności modelu. W porównaniu do CrewAI, który jest wygodny do szybkiego prototypowania, LangGraph oferuje większą elastyczność w zarządzaniu cyklami zwrotnymi i nie ogranicza się do sekwencyjnego wykonywania zadań. Kolejną kluczową zaletą jest lepsze wsparcie dla integracji z zewnętrznymi solverami, poprzez asynchroniczne wywołania i ponawiania prób. Dodatkowo LangGraph umożliwia dynamiczne interakcje z użytkownikiem w trakcie procesu. Rozważaliśmy również Google ADK, ale zrezygnowaliśmy z niego ze względu na silne powiązanie z infrastrukturą Google (taką jak Vertex AI czy Gemini), co mogłoby komplikować dalszy rozwój.

Tabela 1: Porównanie frameworków do implementacji systemu agentowego

Kryterium	**LangGraph**	**CrewAI**	**Google ADK**
Obsługa złożonych przepływów	Natywne wsparcie dla pętli iteracyjnych i warunków decyzyjnych	Ograniczenie do sekwencyjnego wykonywania zadań	Wsparcie dla hierarchicznych systemów wieloagentowych
Elastyczność	Wysoka elastyczność w zarządzaniu cyklami zwrotnymi	Wygodny do szybkiego prototypowania, mniejsza elastyczność	Wysoka elastyczność, modułowa architektura
Integracja z solverami	Lepsze wsparcie dla asynchronicznych wywołań i ponownych prób	Ograniczone możliwości integracji	Szerokie możliwości integracji z narzędziami
Interakcje z użytkownikiem	Dynamiczne interakcje w trakcie procesu (human-in-the-loop)	Wsparcie dla human-in-the-loop	Wsparcie dla human-in-the-loop
Zależności infrastrukturalne	Niezależny od konkretnej infrastruktury	Niezależny od konkretnej infrastruktury	Silne powiązanie z infrastrukturą Google (Vertex AI, Gemini)
Zarządzanie stanem	Scentralizowany system stanów z historią	Podstawowe zarządzanie stanem	Zarządzanie stanem w kontekście Google Cloud
Zastosowanie	Złożone przepływy z iteracjami między agentami	Szybkie prototypowanie, role-based workflows	Aplikacje produkcyjne w ekosystemie Google

2 Architektura systemu

2.1 Przegląd komponentów

System IDE Square LLM składa się z dwóch głównych komponentów:

1. **Multi-Agent System (MAS / BOT)** – niezależny serwis backendowy wykorzystujący duże modele językowe
2. **IDE Square Client** – aplikacja kliencka z interfejsem graficznym

2.2 Multi-Agent System (BOT)

Technologia: Python, FastAPI, LangGraph, OpenAI API, Z3 Solver

Funkcje:

- Analiza wymagań w języku naturalnym
- Budowa modeli formalnych (logika, ontologia, maszyna stanów)
- Orkiestracja agentów wyspecjalizowanych
- Weryfikacja spójności formalizowanych wymagań
- Protokół komunikacji WebSocket (JSON)

Endpointy:

- `WS /workflow` – główne połączenie WebSocket do przetwarzania wymagań

Baza danych: SQLite (sesje, checkpointy LangGraph)

2.3 IDE Square Client

Technologia: Aplikacja webowa z interfejsem graficznym

Funkcje:

- Interfejs graficzny do edycji wymagań
- Wizualizacja maszyny stanów
- Wyświetlanie czatu z asystentem MAS
- Prezentacja modeli logicznych i ontologii
- Klient WebSocket do komunikacji z MAS

2.4 Komunikacja BOT $\leftrightarrow$ IDE

Komunikacja odbywa się przez **protokół WebSocket** z przesyłaniem wiadomości JSON (szczegóły w Sekcji 5).

Dlaczego WebSocket, a nie wywołania lokalne?

- Separacja komponentów – niezależny rozwój MAS i IDE
- Możliwość uruchomienia MAS na oddzielnej maszynie/kontenerze
- Izolacja błędów – awaria MAS nie powoduje awarii IDE
- Strumieniowanie zdarzeń w czasie rzeczywistym

3 Szczegółowy opis workflow

3.1 Orkiestracja agentów

System wykorzystuje głównego agenta LLM Agent jako koordynatora, który decyduje o kolejności uruchamiania pozostałych agentów oraz analizuje postęp w realizacji celu globalnego. Na podstawie analizy danych wejściowych koordynator wybiera jedną z trzech ścieżek:

- `logic` – przekazanie do LogicAgent w celu budowy modeli formalnych
- `clarify` – żądanie doprecyzowania wymagań otrzymanych od użytkownika (ClarificationAgent)
- `complete` – zakończenie procesu

3.2 Sekwencja przetwarzania

Po wyborze ścieżki `logic`, agenty są uruchamiane w następującej kolejności:

$$\text{LLM Agent} \rightarrow \text{Logic Agent} \rightarrow \text{Class Agent} \rightarrow \text{State Agent} \rightarrow \text{Verifier Agent}$$

Po zakończeniu procesu weryfikacji (Verifier Agent) przepływ wraca do agenta głównego (LLM Agent), który może zdecydować o przeprowadzeniu kolejnej iteracji lub zakończeniu procesu.

3.3 Przekazywanie danych między agentami

3.3.1 LLM Agent → Logic Agent

LLM Agent przekazuje ustrukturyzowany zestaw wymagań zawierający:

- Formuły logiczne z typem (A, E, I, O), podmiotem, orzeczeniem i poziomem pewności
- Zidentyfikowane encje
- Wykryte stany i przejścia
- Lista niejednoznaczności

3.3.2 Logic Agent → Class Agent / State Agent

Logic Agent przekazuje:

- Model logiczny (`SquareLogicModel`) zawierający relacje A/E/I/O
- Wstępny model maszyny stanów (`StateMachine`)
- Zidentyfikowane podmioty i relacje
- Listę sprzeczności oraz flagę `is_consistent`

3.3.3 Class Agent ↔ Prover Agent

Class Agent wywołuje Prover Agent w celu formalnej weryfikacji rozłączności klas. Prover Agent zwraca wynik dowodu (`ProofResult`) zawierający:

- Informację o poprawności twierdzenia (`is_valid`)
- Czas wykonania dowodu w milisekundach
- Wyjście z solvera Z3
- Kontrprzykład (jeśli znaleziono naruszenie)

3.3.4 Przejście do Verifier Agent

Verifier Agent otrzymuje wszystkie wygenerowane wcześniej modele:

- Model logiczny
- Ontologię (klasy i właściwości)
- Maszynę stanów

Agent zwraca raport weryfikacji otrzymanych modeli (`VerificationReport`) zawierający status ich spójności, wykryte problemy wraz z poziomami ich istotności (error, warning, info) oraz propozycje ewentualnyhc napraw.

3.4 Pętla iteracyjna

VerifierAgent może wielokrotnie konsultować się z LogicAgent, StateAgent i ProverAgent, aż do osiągnięcia oczekiwanej spójności modelu. Po każdej przeprowadzonej iteracji przepływ wraca do LLM Agent, który ocenia postęp i decyduje o kontynuacji lub zakończeniu procesu.

3.5 Algorytm orkiestracji

3.5.1 Główna pętla workflow

Algorithm 1 Główna pętla orkiestracji MAS

```
state ← WorkflowState(requirements)
iteration ← 0
max_iterations ← 5
while iteration < max_iterations do
    iteration ← iteration + 1
                                                    ▷ Koordynator analizuje obecny stan
    coordination_result ← LLMAgent.coordinate(state)
    route ← determine_route(state)
    if route = "complete" then
        return state                                ▷ Sukces - modele wygenerowane
    else if route = "clarify" then
        state ← ClarificationAgent.execute(state)
    else if route = "logic" then
        state ← LogicAgent.execute(state)
        state ← ClassAgent.execute(state)
        state ← StateAgent.execute(state)
        state ← VerifierAgent.execute(state)
    end if
end while
return state                                        ▷ Przekroczono limit iteracji
```

3.5.2 Algorytm decyzyjny routingu

Funkcja: `determine_route(state) → {"logic", "clarify", "complete"}`

Warunek	Routing	Opis
$iteration \geq max_iterations$	`complete`	Przekroczono limit iteracji
$\neg state.logic_model \wedge iteration = 1$	`logic`	Pierwsza iteracja
$state.clarifications_needed \wedge \neg state.logic_model$	`clarify`	Wymagane doprecyzowanie
$state.logic_model \wedge state.ontology_model \wedge state.state_machine \wedge state.verification_results$	`complete`	Wszystkie modele gotowe
Domyślnie	`logic`	Kontynuuj budowę modeli

Tabela 2: Tabela decyzyjna routingu orkiestratora

3.6 Wynik końcowy

Po zakończeniu przetwarzania system generuje odpowiedź końcową zawierającą:

- `logic_model` – relacje A/E/I/O oraz encje
- `ontology_model` – klasy wraz z ich właściwościami

- `state_machine` – stany, przejścia oraz metadane
- `verification_results` – listę raportów weryfikacji
- `messages` – dziennik komunikatów agentów

4 Agenty

4.1 Komponenty systemu

System wieloagentowy składa się z sześciu wyspecjalizowanych agentów, z których każdy odpowiada za określony aspekt przetwarzania wymagań i budowy modeli formalnych. Agenty współpracują ze sobą, przekazując wyniki swojej pracy kolejnym komponentom systemu.

1. **LLM Agent** – Agent odpowiedzialny za interpretację danych wejściowych przy użyciu modelu językowego oraz ekstrakcję formalnych wymagań logicznych. Pełni również funkcję koordynatora, decydując o kolejności uruchamiania pozostałych agentów.

2. **Logic Agent** – Przetwarza wymagania otrzymane od LLMAgent, tworząc model oparty na kwadracie logicznym oraz wstępny model maszyny stanów. Weryfikuje spójność formuł logicznych i identyfikuje sprzeczności.

3. **State Agent** – Przekształca model logiczny w kompletną maszynę stanów, inteligentnie klasyfikując encje jako stany (z cyklem życia) lub atrybuty (właściwości).

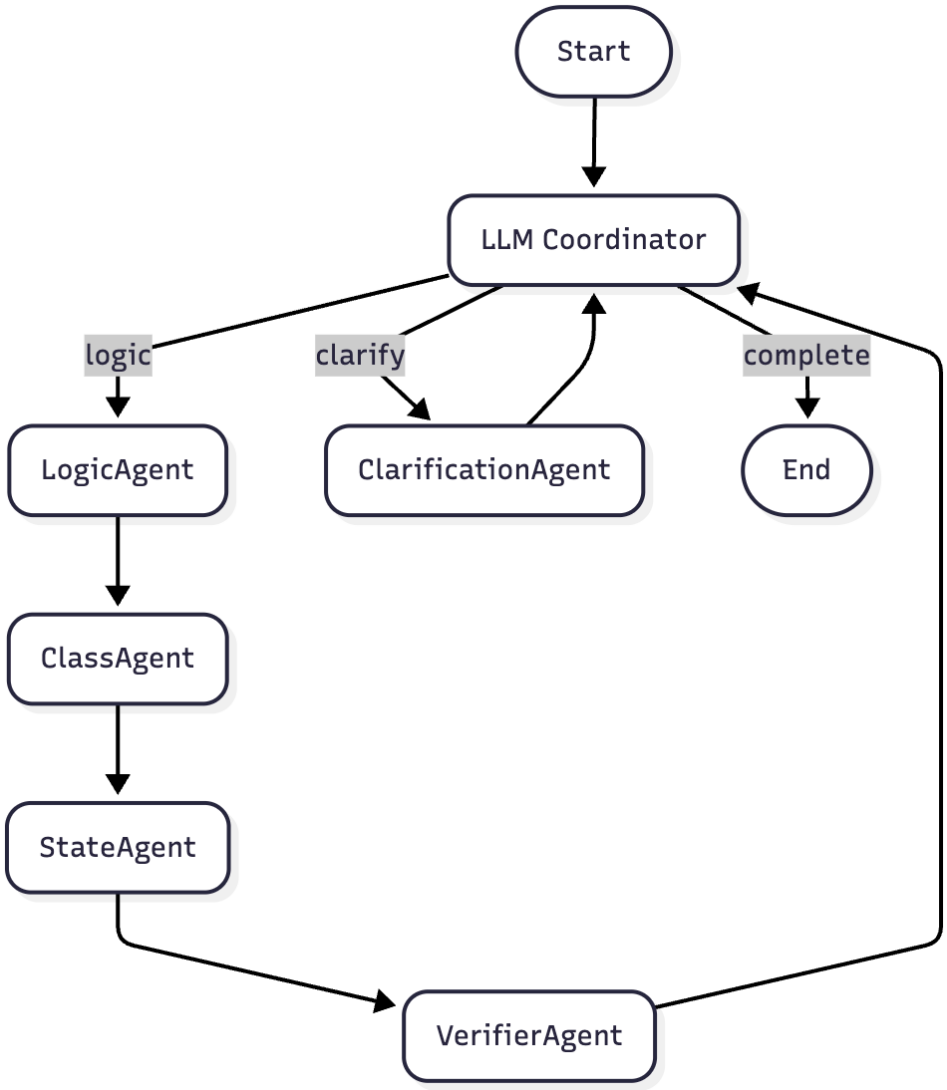
4. **Class Agent** – Transformuje relacje logiczne kwadratu logicznego w strukturalny model ontologii zawierający klasy, właściwości oraz związki między nimi.

5. **Prover Agent** – Agent odpowiedzialny za formalne dowodzenie i weryfikację logiczną przy użyciu solvera Z3.

6. **Verifier Agent** – Przeprowadza kompleksową weryfikację spójności między wszystkimi modelami w systemie, wykrywa niespójności oraz generuje propozycje napraw.

Szczegółowy diagram przepływu między agentami został przedstawiony w kolejnej sekcji.

4.2 Diagram przepływu



Rysunek 1: Diagram przepływu

4.3 LLM Agent

LLM Agent jest wyspecjalizowanym agentem odpowiedzialnym za interpretację danych wejściowych oraz za generowanie ustrukturyzowanych wymagań logicznych, które są zrozumiałe dla innych agentów systemu. Agent pełni również funkcję koordynatora w systemie wieloagentowym. Decyduje on o kolejności uruchamiania poszczególnych agentów oraz analizuje postęp w realizacji celu globalnego.

Główne cele agenta:

- Analiza danych wejściowych przy użyciu modelu językowego LLM.
- Ekstrakcja formalnych wymagań logicznych w oparciu o kwadrat logiczny (A, E, I, O).
- Identyfikacja encji, stanów, przejść i niejednoznaczności w tekście.
- Tworzenie danych wejściowych dla innych agentów.
- Koordynacja współpracy między agentami

- Ocena postępu realizacji zadań.

Agent analizuje dane wejściowe i na ich podstawie tworzy ustrukturyzowany zestaw wymagań w formacie JSON.

Dla każdej formuły logicznej identyfikowane są następujące elementy:

- typ formuły (A, E, I, O),
- podmiot
- orzeczenie

Agent rozpoznaje również potencjalne stany i przejścia. Wyznacza możliwe niejednoznaczności w danych, wymagające doprecyzowania.

Przewidywanie relacji

Agent został rozszerzony dodatkową metodę, która również używa modelu LLM. Na podstawie podanej na wejściu relacji, agent przewiduje pozostałe relacje Kwadratu Logicznego, zgodnie z zasadami sprzeczności, przeciwieństwa i podrzędności. Metoda oczekuje danych wejściowych sformatowanych jako json:

Format wejścia:

Pole	**Opis/Wartości**
`relation.type`	`universal_affirmative` | `universal_negative` | `particular_affirmative` | `particular_negative`
`relation.subject`	Podmiot relacji (ciąg znaków)
`relation.predicate`	Orzeczenie relacji (ciąg znaków)

Tabela 3: Schemat wejścia metody przewidywania relacji

Metoda analizuje kwadrat logiczny, uwzględniając zasady sprzeczności, przeciwieństwa oraz podrzędności. Agent zwraca wynik w postaci:

Format wyjścia - struktura obiektu w tablicy `inferred_relations`:

Pole	**Opis/Wartości**
`type`	Typ relacji logicznej: `universal_affirmative`, `universal_negative`, `particular_affirmative`, `particular_negative`
`square_label`	Oznaczenie w kwadracie logicznym: `A`, `E`, `I`, lub `O`
`status`	Status prawdziwości: `TRUE`, `FALSE`, lub `UNDETERMINED`
`statement`	Formalna sentencja logiczna (np. "All S are P.")
`natural_language`	Parafraza w języku naturalnym

Tabela 4: Struktura pojedynczej wywnioskowanej relacji

Status – przewidywany logiczny status relacji względem danej relacji wejściowej:

- TRUE – relacja wynika logicznie z podanej relacji i jest prawdziwa.

- FALSE – relacja jest sprzeczna z podaną i jest fałszywa.
- UNDETERMINED – agent nie potrafi jednoznacznie określić, czy relacja jest prawdziwa czy fałszywa.

Przykładowo, jeśli A jest prawdą:

- E musi być fałszem
- I jest prawdą (bo A $\rightarrow$ I)
- O jest fałszem

Wyniki przewidywania agenta:
Poniższa tabela przedstawia wyniki przewidywania dla czterech typów relacji wejściowych:

Wej.	Zdanie wejściowe	Label	Status	Natural Language
A	*All cars are red*	E	-	*Agent zwrócił null*
		I	-	
		O	-	
O	Some birds do not fly	A	FALSE	Every bird flies
		E	UNDET.	No birds fly
		I	UNDET.	At least some birds fly
I	Some autonomous vehicles are highway-certified	A	UNDET.	Every vehicle is certified
		E	FALSE	No vehicles are certified
		O	UNDET.	Some vehicles are not certified
E	No reptiles are mammals	A	FALSE	Every reptile is a mammal
		I	FALSE	At least one reptile is a mammal
		O	TRUE	At least one reptile is not a mammal

Tabela 5: Wyniki przewidywania relacji dla różnych typów zdań wejściowych

Uwaga: Dla zdania typu A ("All cars are red") agent nie poradził sobie z przewidywaniem i zwrócił wartość `null`.

Wynik analizy zdań agenta

Agent ma również funkcję pozwalające na przeprowadzenie kompleksowej analizy zdań wejściowych. W tym przykładzie zostały użyte poniższe zdania:

Przykład – Order Management System:

- All new orders are in pending state
- No order in cancelled state is in processing state
- Some orders are priority
- Not all pending orders are priority
- Order can transition from pending to processing
- Order can transition from processing to completed
- Order can be cancelled from any state

Agent zwraca pewność wyników oraz informację o potrzebie doprecyzowania: **Confidence: 0.95**, **Needs clarification: True**

Wykryte formuły logiczne:

#	Typ	Podmiot	Orzeczenie	Źródło	Pewność
1	A	new orders	pending state	All new orders are in pending state	1.0
2	E	cancelled orders	processing state	No order in cancelled state is in processing state	1.0
3	I	orders	priority	Some orders are priority	1.0
4	O	pending orders	priority	Not all pending orders are priority	0.95

Tabela 6: Wykryte relacje kwadratu logicznego

Zidentyfikowane elementy:

Kategoria	Elementy
Encje	order, new order, pending state, processing state, completed state, cancelled state, priority
Stany	pending, processing, completed, cancelled
Przejścia	pending → processing, processing → completed, any → cancelled

Tabela 7: Zidentyfikowane encje, stany i przejścia

Niejednoznaczności:

- Dokładne znaczenie ”priority” jest niejasne (czy to stan, flaga, czy kategoria?)
- Fraza ”Order can be cancelled from any state” nie precyzuje warunków lub ograniczeń anulowania
- Brak jawnych ograniczeń czasowych dla przejść (np. timing, kolejność operacji)

4.4 Logic Agent

Logic Agent jest wyspecjalizowanym agentem odpowiedzialnym za przetwarzanie wymagań pochodzących od Agenta LLM. Na ich podstawie tworzony jest model oparty na kwadracie logicznym oraz wstępny model maszyny stanów.

Agent definiuje słownik mapujący nazwy typów formuł na odpowiadające im typy logiczne:

- universal_affirmative: LogicRelationType.A
- universal_negative: LogicRelationType.E
- particular_affirmative: LogicRelationType.I
- particular_negative: LogicRelationType.O

Główne cele agenta

- Interpretacja i walidacja danych wejściowch (formuły logiczne, podmioty, orzeczenia, stany, przejścia)

- Weryfikacja poprawności otrzymanych formuł logicznych
- Identyfikacja sprzeczności zgodnie z zasadami kwadratu logicznego
- Budowa modelu stanów

Agent sprawdza następujące relacje:

- Przeciwieństwo (A i E) - wymienione zdania nie mogą być zarazem prawdziwe. Gdy, jedno jest prawdziwe, to drugie musi być fałszywe
- Sprzeczność (A–O oraz E–I) - wymienione zdania zawsze mają przeciwne wartości logiczne, jeśli jedno jest prawdziwe, drugie jest fałszywe i na odwrót.
- Podprzeciwieństwo (I i O) - Zdania podprzeciwne nie mogą być zarazem fałszywe. Czyli, gdy jedno jest fałszywe, to drugie musi być prawdziwe.

Agent zwraca strukturę zawierającą:

- Model logiczny (SquareLogicModel)
- Wstępny model stanów (StateMachine)
- Zidentyfikowane podmioty i relacje
- Listę sprzeczności oraz flagę is_consistent

Przykład działania agenta

Agent otrzymuje elementy wykryte przez agenta LLMAgent (opisane w poprzedniej sekcji). Na tej podstawie przeprowadza własną analizę.

Agent oblicza liczbę relacji, podmiotów oraz sprawdza spójność modelu.

```
Number of relations: 4
Number of entities: 11
Consistency: YES
```

Agent wyznacza relacje kwadratu logicznego

```
LOGICAL RELATIONS (Square of Opposition):

A - All X are Y:
  • new orders → pending state (confidence: 1.00)

E - No X are Y:
  • orders in cancelled state → processing state (confidence: 1.00)

I - Some X are Y:
  • orders → priority (confidence: 1.00)

O - Some X are not Y:
  • pending orders → priority (confidence: 0.95)

No contradictions in logic model
```

Agent tworzy wstępny model stanów

```
PRELIMINARY STATE MODEL:

States (4):
  • pending [INITIAL]
  • processing
  • completed
  • cancelled

Transitions (3):
  • pending → processing
  • processing → completed
  • any → cancelled
```

4.5 State Agent

State Agent jest wyspecjalizowanym agentem odpowiedzialnym za przekształcenie modelu logicznego w kompletną maszynę stanów z inteligentną klasyfikacją encji. Agent rozróżnia stany (encje z cyklem życia) od atrybutów (właściwości), co zapewnia logiczną i poprawną semantycznie reprezentację systemu.

Główne cele agenta:

- Inteligentna klasyfikacja encji jako stanów lub atrybutów z wykorzystaniem analizy semantycznej LLM
- Wyznaczenie stanów początkowych i końcowych w maszynie stanów
- Przypisanie atrybutów do stanów na podstawie relacji logicznych (A, E, I, O)
- Generowanie przejść między stanami z odpowiednimi warunkami i poziomami pewności
- Wykrywanie i reprezentowanie wykluczeń (relacje E i O) w metadanych

Klasyfikacja encji:
Agent implementuje dwupoziomową strategię klasyfikacji:

1. **Klasyfikacja semantyczna z LLM (priorytet):** Analiza kontekstowa encji w ramach relacji logicznych, z uwzględnieniem znaczenia domenowego i uzasadnieniem dla każdej decyzji.
2. **Klasyfikacja heurystyczna (fallback):** Analiza morfologiczna (sufiksy przymiotnikowe: -ed, -ing, -able, -ous), częstotliwość występowania jako podmiot vs. orzecznik, wzorce lingwistyczne.

Agent zwraca maszynę stanów zawierającą:

- Listę stanów z ich właściwościami/atrybutami
- Przejścia między stanami z warunkami i poziomami pewności
- Metadane z wykluczeniami i ograniczeniami
- Liczbę wykrytych stanów i przejść

Wynik działania agenta dla przykładu używanego przez wcześniejszych agentów.
Metrics:

- States Count: 4
- Transitions Count: 4
- Initial State: available

States with Properties:

- available (INITIAL)
- cleaning
- reserved
- occupied

Transitions:

- available → reserved
  Condition: a room can transition from available to reserved
  Confidence: 0.95
- reserved → occupied
  Condition: a room can transition from reserved to occupied
  Confidence: 0.95
- occupied → cleaning
  Condition: a room can transition from occupied to cleaning
  Confidence: 0.95
- cleaning → available
  Condition: a room can transition from cleaning to available
  Confidence: 0.95

4.6 Class Agent

Class Agent jest wyspecjalizowanym agentem odpowiedzialnym za przekształcanie relacji logicznych kwadratu logicznego (A/E/I/O) w strukturalny model ontologii zawierający:

- klasy,
- właściwości,
- związki pomiędzy klasami i właściwościami.

Agent analizuje relacje przekazane przez Logic Agenta i na ich podstawie buduje hierarchię klas, wykrywa podklasy oraz ewentualne rozłączności. Klasy są tworzone na podstawie wykrytych podmiotów wyznaczonych z analizowanych relacji. Agent rozpoznaje również predykaty. Właściwości są tworzone na podstawie rozpoznanych predykatów, które nie występują w żadnym zdaniu jako podmiot. Umożliwia to agentowi odróżnianie klas od cech.

Relacje logiczne są tłumaczone w następujący sposób:

- A-relacje (All X are Y) – tworzenie relacji `subClassOf` lub właściwości, w zależności od typu predykatu,
- E-relacje (No X are Y) – generowanie relacji `disjointWith` pomiędzy klasami,

- I-relacje (Some X are Y) – generowanie opcjonalnych właściwości,
- O-relacje (Some X are not Y) – dodawanie adnotacji o wykluczaniu się klas.

Po wygenerowaniu wstępnej ontologii agent przetwarza wszystkie relacje typu E (rozdzielność klas) i przekazuje je do Prover Agenta w celu formalnej weryfikacji poprawności. Dodatkowo Class Agent sprawdza spójność ontologii poprzez wykrywanie cykli w relacjach dziedziczenia oraz konfliktów pomiędzy rozłącznością a dziedziczeniem.

Wynik działania agenta to kompletny model ontologii, gotowy do dalszego przetwarzania. Przykładowe podsumowanie wygenerowanej ontologii przedstawiono poniżej.

Przykład wygenerowanej ontologii

Metryki:

- Liczba klas: 3
- Liczba właściwości: 2
- Sprawdzenia rozłączności: 1
- Wszystkie rozłączności poprawne: ✓

Klasy:

- `premium_room`
- `occupied_room`
  - `excludes_available`
- `room`
  - `partial_exclusion_air_conditioned`

4.7 Prover Agent

Prover Agent jest wyspecjalizowanym agentem odpowiedzialnym za formalne dowodzenie i weryfikację logiczną przy użyciu solvera Z3. Agent zapewnia matematyczną poprawność relacji logicznych poprzez automatyczne wnioskowanie i sprawdzanie spójności modeli opartych na Kwadracie Logicznym.

Główne cele agenta:

- formalna weryfikacja rozłączności klas (disjointness) przy użyciu Z3,
- sprawdzanie spełnialności formuł logicznych (satisfiability checking),
- weryfikacja właściwości Kwadratu Logicznego (contradictory, contrary, subcontrary),
- wykrywanie sprzeczności między relacjami A, E, I, O.

Agent zwraca wynik dowodu (`ProofResult`) zawierający:

- informację, czy twierdzenie jest poprawne (`is_valid`),
- czas wykonania dowodu w milisekundach,
- wyjście z solvera Z3,

- kontrprzykład (jeśli znaleziono naruszenie),
- komunikat błędu (w przypadku niepowodzenia).

Przykład weryfikacji asercji przedstawiono w tabeli 8.

Tabela 8: Przykład weryfikacji asercji przez Prover Agenta

Asercja	**Poprawna**	**Czas (ms)**
`disjoint(occupied_room, available)`	✓	42

4.8 Verifier Agent

VerifierAgent jest wyspecjalizowanym agentem odpowiedzialnym za kompleksową weryfikację spójności między wszystkimi modelami w systemie: modelem logicznym, ontologią oraz maszyną stanów. Agent wykrywa niespójności, błędy strukturalne oraz potencjalne halucynacje (elementy nieobecne w oryginalnych wymaganiach) i generuje propozycje ich naprawy.

Główne cele agenta:

- Weryfikacja spójności logicznej - sprawdzanie reguł Kwadratu Logicznego przy użyciu ProverAgent
- Weryfikacja spójności ontologii - wykrywanie cyklicznych zależności, nieprawidłowych własności
- Weryfikacja spójności maszyny stanów - sprawdzanie osiągalności stanów, poprawności przejść
- Weryfikacja spójności między modelami - zapewnienie zgodności encji logicznych, klas ontologicznych i stanów
- Detekcja halucynacji - wykrywanie elementów nieobecnych w oryginalnych wymaganiach
- Generowanie propozycji napraw dla wykrytych problemów

Agent zwraca raport weryfikacji (VerificationReport) zawierający:

- Status ogólnej spójności (is_consistent)
- Listę wykrytych problemów z poziomami ważności (error, warning, info)
- Listę propozycji napraw wraz z poziomem pewności
- Statystyki: łączna liczba problemów, błędów, ostrzeżeń i napraw
- Metadane o obecności poszczególnych modeli

Przykładowy wynik działania:

- Overall Consistent: ✓
- Issues Found: 0
- Fixes Proposed: 0
- Missing States: 0
- Unreachable States: 0

4.9 Logi

System został wyposażony w mechanizm logowania, który rejestruje wszystkie kluczowe zdarzenia zachodzące podczas przetwarzania. Logowane są:

- Akcje wykonywane przez poszczególnych agentów
- Przejścia między agentami w grafie przepływu
- Komunikaty wymieniane między agentami
- Wywołania zewnętrznych narzędzi (np. solvera Z3)
- Błędy i ostrzeżenia wraz z kontekstem ich wystąpienia

Mechanizm logowania umożliwia późniejszą analizę działania systemu, w tym:

- Debugowanie i identyfikację źródeł błędów
- Analizę wydajności poszczególnych agentów
- Śledzenie przepływu informacji między komponentami
- Audyt i weryfikację poprawności podejmowanych decyzji
- Reprodukcję scenariuszy testowych

5 Integracja z IDE

System wieloagentowy został zintegrowany z aplikacją IDE Square poprzez warstwę serwisową opartą na protokole HTTP. Zdecydowaliśmy się na komunikację sieciową zamiast bezpośredniego wywoływania funkcji Pythona, co zapewnia separację komponentów i umożliwia niezależny rozwój obu części systemu. Takie podejście pozwala również na uruchomienie serwisu MAS na oddzielnej maszynie lub w kontenerze, a w przypadku błędu w systemie agentowym nie dochodzi do awarii całego środowiska IDE.

5.1 Architektura serwisu

Warstwa serwisowa została zaimplementowana przy użyciu frameworka `FastAPI` i udostępnia endpoint WebSocket `/workflow`, który umożliwia dwukierunkową komunikację w czasie rzeczywistym. Dzięki temu rozwiązaniu możliwe jest:

- Strumieniowanie zdarzeń pośrednich (wejście/wyjście agentów, częściowe wyniki)
- Przesyłanie finalnych modeli (logicznego, maszyny stanów, ontologii)
- Prowadzenie interaktywnej konwersacji z możliwością doprecyzowywania wymagań

Serwis wykorzystuje bazę `SQLite` do przechowywania sesji i punktów kontrolnych, co umożliwia wznawianie i iteracyjne doprecyzowywanie wymagań.

5.2 Integracja po stronie IDE

Klient w aplikacji IDE Square komunikuje się z serwisem za pomocą biblioteki `websockets`. Po otrzymaniu komunikatów z serwisu:

- Komunikaty asystenta są wyświetlane w widoku czatu w czasie rzeczywistym
- Stany i przejścia z wygenerowanej maszyny stanów są mapowane na istniejące struktury grafowe IDE i automatycznie wizualizowane
- Podsumowania modelu logicznego i ontologii są prezentowane w panelu bocznym wraz z ewentualnymi ostrzeżeniami walidacyjnymi

Obecna implementacja zastąpiła wcześniejsze rozwiązania AI w IDE Square, które opierały się na modelach Google Gemini.

5.3 Model komunikacji

Komunikacja między systemem wieloagentowym a aplikacją IDE Square odbywa się poprzez dwukierunkowe połączenie WebSocket. Wszystkie wiadomości są przesyłane w formacie JSON.

5.3.1 Protokół wiadomości

1. **Klient → serwer (inicjalizacja):**

```
{"requirements": "<prompt>", "session_id": "<optional>"}
```

2. **Serwer → klient (strumień):**

- `{"type": "session", "session_id": <id>}` – nadany lub potwierdzony identyfikator sesji
- `{"type": "event", "data": <partial_state>}` – kroki pośrednie z LangGraph (bieżące komunikaty, cząstkowe modele)
- `{"type": "final", "data": <final_payload>}` – wynik końcowy
- `{"type": "error", "error": "...", "detail": "..."}` – informacja o błędzie

5.3.2 Struktura odpowiedzi końcowej

Obiekt `final_payload` zawiera następujące pola:

- `session_id` – identyfikator sesji
- `logic_model` – relacje A/E/I/O oraz encje
- `ontology_model` – klasy i właściwości (jeśli zostały wygenerowane)
- `state_machine` – stany (nazwa, opis), przejścia (stan źródłowy, stan docelowy, warunek) oraz opcjonalne metadane
- `verification_results` – lista raportów weryfikacji
- `messages` – dziennik komunikatów agentów (rola i treść)

5.3.3 Sesje i trwałość

Serwer wykorzystuje bazę SQLite z mechanizmem LangGraph sqlite saver do przechowywania stanu sesji. Jeśli klient przekaże identyfikator istniejącej sesji, serwer może zwrócić wynik z pamięci podręcznej. Pozwala to na wznawianie pracy oraz kontynuowanie konwersacji z zachowaniem kontekstu.

5.3.4 Przepływ komunikacji

Typowy przepływ komunikacji przebiega następująco:

1. IDE otwiera połączenie WebSocket i wysyła prompt wraz z opcjonalnym identyfikatorem sesji
2. Serwer nadaje lub potwierdza `session_id` i waliduje konfigurację
3. Serwer przesyła strumień zdarzeń `event` z krokami poszczególnych agentów
4. Po zakończeniu przetwarzania serwer wysyła komunikat `final` z wygenerowanymi modelami
5. IDE renderuje komunikaty w widoku czatu, aktualizuje maszynę stanów i odświeża wizualizacje
6. Kolejne zapytania mogą wykorzystywać ten sam `session_id` w celu kontynuacji lub skorzystania z pamięci podręcznej

6 Eksperymenty

6.1 Eksperyment 1

Eksperyment demonstruje strukturę systemu multi-agentowego, pokazując, jak trzy agenty: LLMAgent, LogicAgent i StateAgent, współpracują w ramach przepływu zdefiniowanego jako graf zależności. Eksperyment pozwala obserwować przepływ informacji między agentami oraz sprawdzić, jak każdy z nich przyczynia się do końcowego modelu systemu.

Dane wejściowe:

```
Hotel reservation system:
   - All premium rooms have a balcony
   - No occupied room is available for reservation
   - Some rooms have an ocean view
   - Not all rooms are air-conditioned

   A room can transition from available to reserved.
   A room can transition from reserved to occupied.
```

Wyniki:

LLMAgent Result

Formulas	Entities	Ambiguities
4	premium rooms, rooms, occupied rooms, balcony, ocean view, air-conditioned, available, reserved	1. The exact meaning of ‘available‘ in *No occupied room is available for reservation* could be clarified: does it mean "available for reservation"or generally available? 2. The statement *Not all rooms are air-conditioned* implies that some rooms lack air-conditioning, but it does not specify how many or which ones. 3. No explicit temporal constraints are provided; it is unclear if transitions may occur repeatedly or under special conditions.

LogicAgent Result

Relations count	Entities found	Is consistent	Contradictions
4	premium rooms, have a balcony, occupied rooms, available for reservation, rooms, have an ocean view, are air-conditioned, balcony, ocean view, air-conditioned, available, reserved	✓	-

StateAgent Result

States	Transitions
occupied, available, reserved	available → reserved: Transition from available to reserved reserved → occupied: Transition from reserved to occupied

Pełna maszyna stanów w formacie JSON:


```
{
  "states": [
    {"name": "occupied", "is_initial": true, "is_final": false},
    {"name": "available", "is_initial": false, "is_final": false},
    {"name": "reserved", "is_initial": false, "is_final": false}
  ],
  "transitions": [
    {"from": "available", "to": "reserved", "confidence": 0.95},
    {"from": "reserved", "to": "occupied", "confidence": 0.95}
  ],
  "initial_state": "occupied"
}
```


Końcowy rezultat

- Extracted requirements: 4
- Logic model entities: premium rooms, have a balcony, occupied rooms, available for reservation, rooms, have an ocean view, are air-conditioned, balcony, ocean view, air-conditioned, available, reserved
- State machine states: 3
- State machine transitions: 2

6.2 Eksperyment 2

Porównanie wygenerowanych maszyn stanowych z modelami referencyjnymi

Przeprowadziliśmy eksperyment, w którym MAS-LLM dostaje krótki opis wymagań (A/E/I/O + relacje) i na jego podstawie ma odtworzyć maszynę stanów. Porównujemy rezultat z przygotowanym modelem wzorcowym.

Wyniki eksperymentów:

Wyniki: order_lifecycle

- F1 States: 0.889 (Próg ≥ 0.80) → ✓
- F1 Transitions: 0.571 (Próg ≥ 0.80) → ✗
- Proof Pass %: 100.0% (Próg $\geq 90\%$) → ✓
- ED Graph (norm): 0.500 (Próg ≤ 0.10) → ✗
- Całość: **FAILED** Czas: 14957ms

Wyniki: document_workflow

- F1 States: 0.667 → ✗
- F1 Transitions: 0.000 → ✗
- Proof Pass %: 100% → ✓
- ED Graph: 1.000 → ✗
- Całość: **FAILED** Czas: 11556ms

Wyniki: user_account

- F1 States: 0.727 → ✗
- F1 Transitions: 0.571 → ✗
- Proof Pass %: 100% → ✓
- ED Graph: 0.667 → ✗
- Całość: **FAILED** Czas: 13500ms

Wyniki: ticket_support

- F1 States: 0.727 → ✗
- F1 Transitions: 0.286 → ✗
- Proof Pass %: 100% → ✓
- ED Graph: 1.000 → ✗
- Całość: **FAILED** Czas: 10306ms

Wyniki: payment_processing

- F1 States: 0.909 → ✓
- F1 Transitions: 0.800 → ✓
- Proof Pass %: 100% → ✓
- ED Graph: 0.300 → ✗
- Całość: **FAILED** Czas: 16347ms

Wyniki: inventory_item

- F1 States: 0.800 → ✓
- F1 Transitions: 0.571 → ✗
- Proof Pass %: 100% → ✓
- ED Graph: 0.556 → ✗
- Całość: **FAILED** Czas: 9819ms

Podsumowanie eksperymentu

- F1 States (Średnia): 0.787 → ✗
- F1 Transitions (Średnia): 0.467 → ✗
- Proof Pass % (Średnia): 100% → ✓
- ED Graph (Średnia): 0.670 → ✗

6.3 Eksperyment 3

Odporność na szum w wymaganiach Kolejnym eksperymentem jest sprawdzenie odporności systemu na szum w wymaganiach. System dostaje ten sam opis w trzech wariantach: czystym, lekko niejednoznacznym oraz bardzo zaszumionym.

Mierzymy:

- F1_stany/przejścia (spadek wraz z szumem),
- Halluc_rate (ile stanów/przejść nie ma pokrycia w tekście),
- Clarif_rounds (ile razy system musiał dopytać/wyjaśnić).

Wyniki: order_management

Poziom Szumu	F1 States	F1 Trans	Halluc %	Clarif	Status
CLEAN	1.000	0.571	0.0%	0	✓
SLIGHT	0.800	0.571	0.0%	0	✓
HEAVY	1.000	0.667	0.0%	0	✓

Spadek F1 (heavy vs clean):

- States: 0.000 (✓ $\leq$ 0.15)
- Transitions: -0.095 (✓ $\leq$ 0.15)

Halucynacje: Clean (✓), Slight (✓) [Wymagany $\leq$ 5%]
Klaryfikacja (Clarification): ✓
Całościowo: **SUCCESS**

Wyniki: hotel_reservation

Poziom Szumu	F1 States	F1 Trans	Halluc %	Clarif	Status
CLEAN	0.750	0.500	0.0%	0	✓
SLIGHT	0.750	0.500	0.0%	0	✓
HEAVY	0.750	0.500	0.0%	0	✓

Spadek F1 (heavy vs clean):

- States: 0.000 (✓ $\leq$ 0.15)
- Transitions: 0.000 (✓ $\leq$ 0.15)

Halucynacje: Clean (✓), Slight (✓) [Wymagany $\leq$ 5%]
Klaryfikacja (Clarification): ✓
Całościowo: **SUCCESS**

Wyniki: traffic_light

Poziom Szumu	F1 States	F1 Trans	Halluc %	Clarif	Status
CLEAN	0.750	1.000	0.0%	0	✓
SLIGHT	0.667	1.000	0.0%	0	✓
HEAVY	0.857	1.000	0.0%	0	✓

Spadek F1 (heavy vs clean):

- States: -0.107 (✓ $\leq$ 0.15)
- Transitions: 0.000 (✓ $\leq$ 0.15)

Halucynacje: Clean (✓), Slight (✓) [Wymagany $\leq$ 5%]
Klaryfikacja (Clarification): ✓
Całościowo: **SUCCESS**

Podsumowanie Eksperymentu E3

- Średni Spadek F1 (heavy vs clean):
  - States: -0.036 (Próg $\leq$ 0.15) $\rightarrow$ ✓
  - Transitions: -0.032 (Próg $\leq$ 0.15) $\rightarrow$ ✓
- Wskaźnik Halucynacji (Clean): 0.0% (Próg $\leq$ 5%) $\rightarrow$ ✓
- Wskaźnik Halucynacji (Slight): 0.0% (Próg $\leq$ 5%) $\rightarrow$ ✓
- Wskaźnik Halucynacji (Heavy): 0.0% (Próg: brak) $\rightarrow$ -
- Średnie Rundy Klaryfikacji: 0.0 (Próg $\leq$ 3) $\rightarrow$ ✓

6.4 Eksperyment 4: Porównanie podejść

Eksperyment porównuje trzy podejścia do generowania maszyn stanów z opisu tekstowego, ze szczególnym uwzględnieniem zdolności do wykrywania i respektowania ograniczeń wynikających z kwadratu logicznego. Celem jest sprawczenie, czy podejście wieloagentowe (S) osiąga wyższą spójność logiczną niż metody bazowe (B1, B2).

6.4.1 Podejścia

B1: SingleLLM

Podejście B1 opiera się na bezpośrednim wykorzystaniu modelu językowego (GPT-4.1-mini) do generowania kompletnej maszyny stanów. Proces może odbywać się iteracyjnie (do 5 prób), przy czym informacja zwrotna oparta jest wyłącznie na heurystykach strukturalnych: minimalnej liczbie stanów ($\geq$ 3), przejść ($\geq$ 2) oraz obecności stanu początkowego i końcowego.

Podejście to **nie wykorzystuje** zasad kwadratu logicznego. Może więc generować przejścia naruszające ograniczenia typu E („No X is Y”), których nie potrafi samodzielnie wykryć.

B2: ManualSquare

Podejście B2 jest w pełni deterministyczne, bez użycia modeli językowych. Stany identyfikowane są przez dopasowanie słów kluczowych (*pending*, *active*, *completed*, *cancelled*, itp.), a następnie walidowane regułami kwadratu logicznego.

Eliminowane są pary sprzeczne, takie jak *active/inactive* czy *approved/rejected*, zgodnie z zasadą kontradikcji. Proces iteracyjny (do 5 prób) kończy się po osiągnięciu stabilności. Metoda cechuje się wysoką przewidywalnością, lecz jest ograniczona do zdefiniowanego słownika, nie rozumie semantyki nowych terminów.
S: SquareBotApproach

Podejście S wykorzystuje pełny potok wieloagentowy składający się z:

1. **LLMAgent** — ekstrakcja wymagań z opisu tekstowego
2. **LogicAgent** — budowa modelu logicznego z formułami A/E/I/O
3. **StateAgent** — generowanie maszyny stanów
4. **VerifierAgent** — weryfikacja spójności logicznej

Kluczową różnicą jest jawna analiza formuł kwadratu logicznego przez LogicAgent, który wykrywa ograniczenia typu E („No cancelled order can be shipped") i zapewnia, że wygenerowana maszyna stanów ich nie narusza. Proces może być iterowany (do 5 razy) w przypadku wykrycia niespójności przez VerifierAgent.

6.4.2 Metryki oceny

F1 Score
Miara poprawności strukturalnej - porównanie wygenerowanych stanów z referencyjnymi:

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

Logical Consistency
Miara zgodności z regułami kwadratu logicznego. Sprawdzane są formuły:

- **A** (Universal Affirmative): „All X are Y"
- **E** (Universal Negative): „No X is Y" — kluczowe!
- **I** (Particular Affirmative): „Some X are Y"
- **O** (Particular Negative): „Some X are not Y"

Każde naruszenie (np. przejście $cancelled \rightarrow shipped$ mimo formuły E) obniża wynik o 15%.
Disjointness
Sprawdzenie, czy stany są wzajemnie wykluczające się (obiekt może być tylko w jednym stanie).
Wynik końcowy

$$Score = 0.4 \cdot F1 + 0.4 \cdot LogicalConsistency + 0.2 \cdot Disjointness$$

6.4.3 Dane testowe

Eksperyment wykorzystuje 6 przypadków testowych z różnych domen biznesowych. Każdy przypadek zawiera opis wymagań w języku naturalnym z formułami kwadratu logicznego oraz model maszyny stanów.
Przypadek 1: order_lifecycle

Wymagania (dane wejściowe):

Order processing system:

- All new orders are pending (A)
- All pending orders can become processing (A)
- All processing orders can become completed (A)
- No cancelled order is processing (E)
- No completed order is pending (E)
- Some orders are priority (I)
- Not all orders are completed (O)

Oczekiwane stany: pending, processing, completed, cancelled
Wyniki:

Podejście	F1	Logic %	Sprzeczności	Disjoint
B1 (SingleLLM)	1.00	72%	1	✓
B2 (ManualSquare)	0.89	72%	1	✓
S (SquareBot)	0.80	**100%**	**0**	✓

Analiza: Podejście S jako jedyne osiągnęło 100% spójności logicznej. B1 mimo instrukcji w prompcie wygenerowało jedno zabronione przejście.
Przypadek 2: document_workflow

Wymagania (dane wejściowe):

Document management workflow:

- All new documents are in draft state (A)
- All draft documents require review before approval (A)
- No rejected document is approved (E)
- No approved document is in draft (E)
- All reviewed documents are either approved or rejected (A)
- Some documents require multiple reviews (I)
- Not all documents are approved (O)

Oczekiwane stany: draft, review, approved, rejected
Wyniki:

Podejście	F1	Logic %	Sprzeczności	Disjoint
B1 (SingleLLM)	0.75	37%	2	×
B2 (ManualSquare)	0.67	37%	2	×
S (SquareBot)	0.67	37%	2	✓

Analiza: Najtrudniejszy przypadek; żadne podejście nie osiągnęło wysokiej spójności logicznej. Jednak S jako jedyne zachowało poprawną strukturę rozłączności stanów. Wskazuje to na potrzebę dalszego rozwijania ekstrakcji formuł logicznych.

6.4.4 Przypadek 3: user_account

Wymagania (dane wejściowe):

User account management:

- All new accounts are inactive (A)
- All inactive accounts can be activated (A)
- No suspended account is active (E)
- No deleted account is active or suspended (E)
- All active accounts can be suspended (A)
- Some accounts are premium (I)
- Not all accounts are verified (O)

Oczekiwane stany: inactive, active, suspended, deleted
Wyniki:

Podejście	F1	Logic %	Sprzeczności	Disjoint
B1 (SingleLLM)	1.00	37%	2	×
B2 (ManualSquare)	0.67	68%	1	✓
S (SquareBot)	0.73	**100%**	**0**	×

Analiza: Znacząca różnica w spójności logicznej: S osiągnęło 100%, podczas gdy B1 tylko 37% mimo wysokiego F1. Pokazuje to, że dopasowanie stanów (F1) nie gwarantuje poprawności logicznej.

6.4.5 Przypadek 4: ticket_support

Wymagania (dane wejściowe):

Support ticket system:

- All new tickets are open (A)
- All open tickets can be assigned (A)
- No closed ticket is open (E)
- No resolved ticket is open (E)
- All assigned tickets are being worked on (A)
- Some tickets are escalated (I)
- Not all tickets are resolved (O)

Oczekiwane stany: open, assigned, resolved, closed
Wyniki:

Podejście	F1	Logic %	Sprzeczności	Disjoint
B1 (SingleLLM)	0.89	71%	1	✓
B2 (ManualSquare)	0.67	71%	1	✓
S (SquareBot)	0.73	71%	1	✓

Analiza: Prosty przypadek; wszystkie podejścia osiągnęły podobne wyniki. Przy niskiej złożoności różnice między metodami są mniej widoczne.

6.4.6 Przypadek 5: payment_processing

Wymagania (dane wejściowe):

Payment processing:

- All new payments are pending (A)
- All pending payments are either authorized or declined (A)
- No refunded payment is pending (E)
- No captured payment is declined (E)
- All authorized payments can be captured (A)
- All captured payments can be refunded (A)
- Some payments are flagged for review (I)
- Not all payments are captured (O)

Oczekiwane stany: pending, authorized, declined, captured, refunded
Wyniki:

Podejście	F1	Logic %	Sprzeczności	Disjoint
B1 (SingleLLM)	0.91	71%	1	✓
B2 (ManualSquare)	0.62	71%	1	✓
S (SquareBot)	0.83	71%	1	✓

Analiza: Złożony przypadek z wieloma ścieżkami. S osiągnęło drugie najlepsze F1 (0.83) po B1 (0.91), jednocześnie zachowując taką samą spójność logiczną.

6.4.7 Przypadek 6: inventory_item

Wymagania (dane wejściowe):

Inventory management:

- All new items are in stock (A)
- No reserved item is available (E)
- No shipped item is in stock (E)
- All in-stock items can be reserved (A)
- All reserved items can be shipped (A)
- Some items are backordered (I)
- Not all items are shipped (O)

Oczekiwane stany: in_stock, reserved, shipped, backordered
Wyniki:

Podejście	F1	Logic %	Sprzeczności	Disjoint
B1 (SingleLLM)	0.75	**100%**	0	✓
B2 (ManualSquare)	0.67	68%	1	✓
S (SquareBot)	0.80	**100%**	0	✓

Analiza: Zarówno B1 jak i S osiągnęły 100% spójności logicznej. Jednak S miało lepsze F1 (0.80 vs 0.75), co wskazuje na lepsze wykrycie stanów.

6.4.8 Podsumowanie wyników

Wyniki zagregowane

Tabela 9: Średnie wyniki dla wszystkich przypadków testowych

Podejście	Avg F1	Avg Logic	Avg Disjoint	Avg Iters	Score
B1 (SingleLLM)	0.88	64.5%	93%	1.0	0.80
B2 (ManualSquare)	0.70	64.5%	97%	3.7	0.73
S (SquareBot)	**0.76**	**79.7%**	97%	1.0	**0.81**

Wzór oceny końcowej:

$$Score = 0.4 \cdot F1 + 0.4 \cdot LogicalConsistency + 0.2 \cdot Disjointness$$

6.5 Eksperyment 5: Ablacja algorytmu

Celem eksperymentu jest zbadanie wpływu poszczególnych komponentów systemu MAS-LLM na jakość generowanych maszyn stanowych. Poprzez systematyczne wyłączanie kolejnych komponentów (ablacja), możemy określić ich rzeczywisty wkład w końcowy rezultat.

6.5.1 Warianty ablacyjne

ID	Wariant	Opis
V1	Full System	Pełny pipeline MAS-LLM (baseline)
V2	No Prover	Bez weryfikacji Z3 (ProverAgent wyłączony)
V3	No Verifier	Bez sprawdzania spójności (VerifierAgent wyłączony)
V4	No Square	Bez dopełniania kwadratu opozycji

Tabela 10: Warianty ablacyjne w eksperymencie E4

6.5.2 Metryki

- **F1 States** – dopasowanie stanów do referencyjnych
- **Contradictions** – liczba wykrytych sprzeczności logicznych
- **Models Rejected** – liczba odrzuconych modeli przez verifier
- **Iterations** – liczba iteracji do stabilnego wyniku
- **Component Impact** – spadek F1 przy wyłączeniu komponentu

6.5.3 Wyniki eksperymentu

Tabela 11: Zagregowane wyniki dla 6 modeli referencyjnych

Wariant	Avg F1	Avg Contradictions	Avg Rejected	Avg Iters
V1 Full System	0.76	0.0	0.0	1.0
V2 No Prover	0.76	0.0	0.0	2.0
V3 No Verifier	0.76	0.0	0.0	1.0
V4 No Square	0.76	0.0	0.0	1.0

6.5.4 Wpływ komponentów

Tabela 12: Wpływ wyłączenia komponentu na F1

Komponent	F1 Impact	Znaczenie
Prover	+0.000	Niskie
Verifier	+0.000	Niskie
Square Completion	+0.000	Niskie

6.5.5 Wyniki szczegółowe per model

Tabela 13: F1 States dla każdego modelu i wariantu

Model	V1 Full	V2 No Prover	V3 No Verifier	V4 No Square
order_lifecycle	0.80	0.80	0.80	0.80
document_workflow	0.67	0.67	0.67	0.67
user_account	0.73	0.73	0.73	0.73
ticket_support	0.73	0.73	0.73	0.73
payment_processing	0.83	0.83	0.83	0.83
inventory_item	0.80	0.80	0.80	0.80

6.5.6 Analiza wyników

Wyniki eksperymentu ablacyjnego są zaskakujące — wszystkie warianty osiągnęły identyczne F1 = 0.76. Oznacza to, że:

1. **LLM jest głównym źródłem jakości** – ekstrakcja wymagań przez model językowy determinuje 76% poprawności stanów.

2. **Prover/Verifier działają jako „strażnicy”** – komponenty walidacyjne nie poprawiają F1 w tych testach, ponieważ dane testowe nie zawierają sprzeczności wymagających odrzucenia.

3. **Wartość komponentów ujawni się przy błędnych danych** – przy wymaganiach ze sprzecznościami lub niejednoznacznościami Prover i Verifier powinny odrzucać niepoprawne modele.

Warto zauważyć, że wariant V2 (No Prover) wymagał średnio 2 iteracji zamiast 1, co wskazuje na różnicę w logice pętli weryfikacyjnej.

6.5.7 Wnioski

- Dla „czystych” wymagań komponenty walidacyjne nie zmieniają wyniku
- Prover i Verifier są istotne dla wykrywania błędów, nie poprawy jakości
- Przyszłe eksperymenty powinny zawierać celowo błędne wymagania

6.6 Eksperyment 6: Dokładność dopełniania kwadratu opozycji

Celem eksperymentu jest zbadanie, jak poprawnie system dopełnia kwadrat logiczny z częściowych danych wejściowych. Testujemy scenariusze, w których podajemy 1, 2 lub 3 rogi kwadratu, a system musi wywnioskować pozostałe relacje.

6.6.1 Kwadrat logiczny

Klasyczny kwadrat logiczny definiuje cztery typy relacji logicznych:

- **A** (Universal Affirmative): „Wszystkie S są P”
- **E** (Universal Negative): „Żadne S nie jest P”
- **I** (Particular Affirmative): „Niektóre S są P”
- **O** (Particular Negative): „Niektóre S nie są P”

Relacje między rogami:

- A i O są sprzeczne (contradictory)
- E i I są sprzeczne (contradictory)
- A i E są przeciwne (contrary)
- I i O są podprzeciwne (subcontrary)

6.6.2 Scenariusze testowe

Scenariusz	Rogi podane	Do wnioskowania	Liczba testów
C1 (1 corner)	1 róg (A, E, I lub O)	3 pozostałe	24
C2 (2 corners)	2 rogi	2 pozostałe	36
C3 (3 corners)	3 rogi	1 pozostały	24

Tabela 14: Scenariusze eksperymentu E5

6.6.3 Przypadki testowe

Eksperyment wykorzystuje 6 predefiniowanych przypadków z różnych domen:

1. `order_complete` – zamówienia e-commerce (A = TRUE)
2. `cancelled_not_active` – anulowane zamówienia (E = TRUE)
3. `some_priority` – priorytetowe zamówienia (I = TRUE)
4. `not_all_shipped` – częściowa wysyłka (O = TRUE)
5. `active_verified` – konta użytkowników (A = TRUE)
6. `rejected_not_approved` – dokumenty workflow (E = TRUE)

6.6.4 Wyniki eksperymentu

Tabela 15: Dokładność wnioskowania dla każdego scenariusza

Scenariusz	Rogi podane	Testy	Relation Accuracy	Status Accuracy
C1 (1 corner)	1	24	29.2%	29.2%
C2 (2 corners)	2	36	76.4%	76.4%
C3 (3 corners)	3	24	91.7%	91.7%

6.6.5 Szczegółowe statystyki

Tabela 16: Poprawne wnioskowania vs całkowita liczba

Scenariusz	Poprawne	Całkowite	Accuracy
C1 (1 corner)	21	72	29.2%
C2 (2 corners)	55	72	76.4%
C3 (3 corners)	22	24	91.7%

6.6.6 Trend dokładności

Wyniki pokazują wyraźny trend — im więcej rogów kwadratu jest podanych, tym wyższa dokładność wnioskowania:

Tabela 17: Trend dokładności wnioskowania

Rogi podane	Accuracy	Trend
1 corner	29.2%	Niska
2 corners	76.4%	Dobra
3 corners	91.7%	Wysoka

6.6.7 Analiza wyników

1. **C1 (1 corner) – niska dokładność (29.2%)**

   Wnioskowanie z pojedynczego rogu jest najtrudniejsze, ponieważ nie wszystkie relacje są jednoznacznie determinowane. Na przykład, jeśli I jest prawdziwe, to A może być prawdziwe lub nieokreślone.

2. **C2 (2 corners) – dobra dokładność (76.4%)**

   Dwa rogi znacząco ograniczają możliwe kombinacje. Szczególnie pary sprzeczne (A+O, E+I) dają silne ograniczenia.

3. **C3 (3 corners) – wysoka dokładność (91.7%)**

   Trzy rogi niemal jednoznacznie determinują czwarty. Pozostałe 8.3% błędów może wynikać z przypadków, gdzie status jest „UNDETERMINED”.

6.6.8 Wnioski

- Model LLM poprawnie rozumie zasady kwadratu opozycji
- Dokładność rośnie wraz z ilością dostarczonych informacji
- Wnioskowanie z 3 rogów osiąga 91.7% dokładności

- Przypadki z 1 rogiem wymagają lepszej obsługi statusu „UNDETERMINED”

7 Wnioski

W ramach zrealizowanego projektu opracowano i przebadano system wieloagentowy, służący do automatycznego generowania maszyn stanów z wymagań tekstowych z uwzględnieniem ograniczeń logicznych. Przeprowadzone eksperymenty pozwoliły na sformułowanie następujących wniosków:

7.1 Skuteczność architektury wieloagentowej

Zastosowanie architektury opartej na wyspecjalizowanych agentach (LogicAgent, VerifierAgent, ProverAgent) pozwoliło na osiągnięcie znacznie wyższej spójności logicznej generowanych modeli (średnio 79.7%) w porównaniu do klasycznego podejścia wykorzystującego pojedynczy model LLM (64.5%). Integracja formalnej weryfikacji (solver Z3) z regułami kwadratu opozycji skutecznie ograniczyła występowanie sprzeczności oraz potencjalnych halucynacji.

7.2 Odporność na jakość danych wejściowych

System wykazał wysoką odporność na zaszumienie pojawiające się w danych wejściowych. Eksperymenty potwierdziły, że nawet przy znacznym poziomie niejednoznaczności w tekście (Heavy Noise), jakość strukturalna i logiczna generowanych modeli pozostaje stabilna, a system nie generuje fałszywych stanów ani przejść.

7.3 Potencjał rozwoju

Wyniki wskazują na możliwość dalszego rozwoju systemu, szczególnie w obszarze ekstrakcji przejść (Transitions), gdzie wyniki były widocznie niższe niż w przypadku wykrywania stanów. Dodatkowo, osiągnięto wysoką skuteczność wnioskowania logicznego na podstawie niepełnych danych (Eksperyment 6). Pokazuje to użyteczność systemu nawet w zadaniach wymagających uzupełniania brakującej wiedzy domenowej.

Podsumowując, projekt potwierdził, że wykorzystanie LLM wraz z metodami formalnymi pozwala na tworzenie bardziej wiarygodnych i deterministycznych narzędzi wspierających procesy inżynierii oprogramowania, przy jednoczesnym zachowaniu kontroli nad poprawnością logiczną generowanych wyników. Należy jednak zauważyć, że skuteczność zaproponowanego podejścia zależy od jakości formalizacji wiedzy oraz zakresu reguł weryfikacyjnych.